CSE 4630: Containerization for Developers

Department of Computer Science and Information Technology

7th Semester

Assignment 3

Docker Images and Dockerfiles

Book Reference	Chapters 6–9

Practical Questions

Q. No.	Question
1	Create the following three containers: docker run -d --name web-server -p 8081:80 nginx docker run -d --name database mysql:8 docker run -d --name cache redis Use appropriate Docker commands to determine for each container: Container ID, container name, image used, current status, published ports, and container IP address. Also display only currently running containers and then display all containers, including stopped containers.
2	Create an Nginx container: docker run -d --name college-web -p 8085:80 nginx Now create another container with the incorrect mapping: docker run -d --name college-web-error -p 8085:8085 nginx The Nginx service is running inside the container, but http://localhost:8085 cannot be accessed. Use docker ps, docker logs, docker inspect, and docker port to identify the problem. Explain the cause and recreate the container with the correct port mapping.
3	Create: docker run -d --name container-a nginx docker run -d --name container-b nginx Enter container-a and generate continuous CPU activity using an appropriate Linux command. Leave container-b running normally. Use Docker resource-monitoring facilities to observe both containers. Implement the following: monitor CPU and memory; identify which consumes more CPU; compare memory usage; stop the CPU-intensive process; and observe the resource usage after stopping it.
4	Run: docker run -it --name alpine-test alpine Inside the container: display the Alpine version; check whether curl is available; install curl

	using apk; retrieve https://example.com with curl; save the content to a file; and display the file. Exit the container and compare the Alpine image size with an Ubuntu image. Explain why Alpine is useful when a small image is desirable.
5	Create a Dockerfile: FROM alpine:latest RUN apk add --no-cache curl WORKDIR /app CMD ["sh"] Build it as alpine-curl:v1, run it, and verify curl. Create an equivalent Ubuntu Dockerfile: FROM ubuntu:latest RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/* WORKDIR /app CMD ["bash"] Build it as ubuntu-curl:v1. Compare image sizes and explain the practical implication.
6	Create: college-website/ ├── index.html └── Dockerfile Use this index.html: <!DOCTYPE html> <html> <head><title>CSE Department</title></head> <body> Department of Computer Science and IT Welcome to the CSE Department. Course: Containerization for Developers </body> </html> Use this Dockerfile: FROM nginx:alpine COPY index.html /usr/share/nginx/html/index.html Build the image as cse-website:v1. Run it with Nginx exposed on host port 8081. Access it from a browser and verify that the custom page is served from the container.
7	Create backend1/index.html containing: Student Service This page is served by Backend 1. Create backend2/index.html containing: Faculty Service This page is served by Backend 2. Run two Nginx containers named student-service and faculty-service. Create an Nginx reverse-proxy configuration so /student routes to student-service and /faculty routes to faculty-service.

	The user should access only the reverse proxy. Implement and verify both routes.
8	Create a user-defined network named college-network. Run frontend, student-service, and faculty-service on this network. The two backend containers should serve simple Nginx webpages. From frontend, use an appropriate command-line utility to communicate with student-service and faculty-service without using IP addresses. Verify network membership, resolve the backend names, access both services using their names, and inspect the network to identify connected containers.

9	Create:
	node-app/
	├── package.json
	├── server.js
	└── Dockerfile
	
	package.json:
	{
	"name": "student-app",
	"version": "1.0.0",
	"main": "server.js",
	"scripts": {"start": "node server.js"},
	"dependencies": {"express": "^4.18.2"}
	}
	
	server.js:
	const express = require("express");
	const app = express();
	app.get("/", (req,res) => res.send("Student Application is running"));
	app.get("/students", (req,res) => res.json([
	{id:1,name:"Student A"}, {id:2,name:"Student B"}
	]));
	app.listen(3000, () => console.log("Server running on port 3000"));
	
	Create a Dockerfile, build and run it as student-app. Create an Nginx reverse proxy and configure the architecture so users access the application through Nginx while Node.js is not directly exposed to the host. Verify / and /students through Nginx.

10	Use the Node.js + Nginx architecture from Question 9. Deliberately introduce one failure in the Node.js backend: stop the container, disconnect it from the network, or configure Nginx with an incorrect backend port. Keep Nginx running. Diagnose the problem using docker ps, docker logs, docker inspect, docker network inspect, and docker port. Determine whether the failure is due to container state, application process, port configuration, or network configuration. Correct the problem and demonstrate that the application works again.